

E-Ponto - Roteiro de Apresentacao

Controle de ponto digital em Flask (REP-P / Portaria MTP n. 671/2021)

UVV - Programacao Avancada para Web

Integrantes: Lucas Andre, Roberto Dias, Rychard Chaves

Este roteiro divide a apresentacao em **3 partes** (uma por integrante) e percorre os **7 criterios tecnicos** da avaliacao na ordem oficial (1 a 7), indicando em cada ponto quem fala, qual arquivo abrir e o que destacar. Tempo sugerido: ~12 a 15 minutos + demonstracao ao vivo.

Divisao em 3 partes

Parte	Responsavel	Tema	Criterios
1	Lucas Andre	Arquitetura e Organizacao	1 (App Factory), 2 (Blueprints)
2	Roberto Dias	Dados e Regras de Negocio	3 (Formularios), 6 (Persistencia/ORM)
3	Rychard Chaves	Interface, Testes e Qualidade	4 (Templates), 5 (Testes), 7 (Qualidade)

A divisao e tematica (cada pessoa domina um bloco coerente), mas a apresentacao segue a ordem dos criterios 1 a 7, alternando os apresentadores conforme indicado abaixo.

Abertura (~30s) - Lucas Andre

“Boa noite. Somos o Grupo [X] e vamos apresentar o **E-Ponto**, uma plataforma web de controle de ponto digital para empresas de ate 200 funcionarios. Diferente de um CRUD simples, o E-Ponto implementa as exigencias legais do REP-P da Portaria 671/2021: numero sequencial de registro, cadeia de hashes anti-fraude e geolocalizacao. Foi feito em **Flask**, e vamos mostrar como ele atende cada criterio tecnico.”

Parte 1 - Arquitetura e Organizacao (Lucas Andre)

Criterio 1 - Arquitetura Modular e Application Factory

Quem fala: Lucas Andre | **Arquivos:** app.py

Fala: "O coracao da arquitetura e o padrao Application Factory: em vez de criar o Flask no nivel do modulo, temos a funcao create_app(test_config). Isso permite criar instancias diferentes - uma para producao, outra para os testes com banco em memoria - e evita import circular entre extensoes e modelos."

O que mostrar:

- A funcao create_app() e a ordem de inicializacao (app.py).
- Cada extensao segue a convencao init_app(app): config -> db + register_models() -> auth -> migrate -> mail -> wtf/limiter -> debugtoolbar -> views.
- CSRF e Rate-Limiter so ativam FORA do modo de teste (decisao consciente para nao atrapalhar os testes).
- register_models() garante que o SQLAlchemy enxergue todas as tabelas antes de qualquer operacao de schema.

Frase de impacto: *A fabrica desacopla a montagem da aplicacao do seu uso - e o que torna possivel testar tudo isso de forma isolada.*

Criterio 2 - Organizacao em Modulos / Blueprints / Rotas

Quem fala: Lucas Andre | **Arquivos:** E_Ponto/ext/, E_Ponto/views/ init .py

Fala: "A responsabilidade e separada por camadas e por dominio. O pacote E_Ponto/ tem ext/ (extensoes), models/ (dados), forms/ (validacao), views/ (rotas) e utils/ (regras de negocio). Sao 6 Blueprints, um por area do sistema."

O que mostrar:

- Mostrar a estrutura de pastas (tree ou print).
- Os 6 Blueprints em views/__init__.py: auth (/auth), main (/), ponto (/ponto), admin (/admin), rh (/rh), funcionario (/funcionario).
- O views/__init__.py tambem centraliza os error handlers (403/404/500) e os filtros Jinja de fuso horario.

Frase de impacto: *Se amanha o RH precisar de uma tela nova, sabemos exatamente onde mexer - views/rh.py - sem tocar no resto.*

Parte 2 - Dados e Regras de Negocio (Roberto Dias)

Criterio 3 - Formularios e Validacao

Quem fala: Roberto Dias | **Arquivos:** E_Ponto/forms/, views/ponto.py

Fala: “Usamos Flask-WTF / WTForms para validacao no servidor - nunca confiamos so no navegador. Cada formulario e uma classe com seus validadores.”

O que mostrar:

- forms/ponto.py (BaterPontoForm): SelectField de tipo, latitude/longitude/precisao como HiddenField (preenchidos por JS), local_trabalho_id com coerce=int, justificativa com Length(max=255).
- Na view, form.validate_on_submit() so executa a logica se os dados forem validos (views/ponto.py).
- Em producao, o CSRF protege todos os POSTs; nos testes ele e desligado de proposito.

Frase de impacto: *Validacao no servidor e seguranca; o front pode mentir, o backend nao deixa.*

Criterio 6 - Persistencia de Dados e ORM

Quem fala: Roberto Dias | **Arquivos:** E_Ponto/models/, ext/db.py, migrations/

Fala: “Persistencia com SQLAlchemy 2.x no estilo moderno Mapped/mapped_column. Sao 14 entidades, com SQLite em dev e PostgreSQL em producao - trocamos so pela variavel de ambiente.”

O que mostrar:

- Diagrama do modelo: User-RoleUser-Role (papeis multi-empresa), Business, LocalTrabalho, Jornada, EscalaFuncionario, Registro, Retificacao, BancoHoras, AuditLog, NsrSequencia.
- models/registro.py: TipoRegistro como Enum, timestamp_utc sempre em UTC, geolocalizacao Numeric(10,7), suspeito_geo.
- Cadeia de hash: hash_registro + hash_anterior (integridade anti-fraude); UniqueConstraint(empresa_id, nsr).
- relationship com foreign_keys explicito (3 FKs para users).
- Migrations com Flask-Migrate/Alembic - o schema e versionado, nao criado na mao.

Frase de impacto: *O hash em cadeia significa que alterar uma batida antiga exigiria refazer todos os hashes seguintes - e a exigencia anti-adulteracao do REP-P, implementada de verdade.*

Demo sugerida (fluxo completo, views/ponto.py): bater um ponto -> gera NSR atomico -> calcula hash -> checa geofence -> salva -> registra no AuditLog -> redireciona pro comprovante (PDF via ReportLab).

Parte 3 - Interface, Testes e Qualidade (Rychard Chaves)

Criterio 4 - Interface e Templates / Frontend

Quem fala: Rychard Chaves | **Arquivos:** templates/

Fala: “O frontend usa Jinja2 + Bootstrap 5, com heranca de templates. Tudo herda de templates/base.html, entao navbar, mensagens flash e responsividade sao centralizados.”

O que mostrar:

- Organizacao de templates/ por dominio: auth/, ponto/, rh/, admin/, funcionario/, errors/, emails/.
- base.html e como as telas estendem dele.
- Filtros customizados de fuso: `{{ dt|localdt('%H:%M') }}` (UTC -> horario local).
- Templates de e-mail (emails/retificacao_decidida) em HTML + texto.
- Paginas de erro proprias (403/404/500) que mantem o visual do sistema.

Frase de impacto: *O funcionario bate ponto pelo celular - por isso a responsividade e testada, nao so assumida.*

Criterio 5 - Testes Automatizados

Quem fala: Rychard Chaves | **Arquivos:** tests/, tests/conftest.py

Fala: “Temos 76 testes automatizados com pytest, organizados por area. Nao e so 'a home abre' - testamos calculos de horas, seguranca e regras de negocio.”

O que mostrar:

- Suite: test_unitarios (19), test_invasao (11), test_acesso (7), test_auth (6), test_ponto/test_calculos (8) e outros - 76 no total.
- conftest.py: fixture app com SQLite em memoria (StaticPool); fixture org cria empresa completa (papeis, usuarios, jornada, local com geofence); fixtures login e criar_registro.
- TZ='UTC' fixo no topo -> calculos de hora deterministicos no CI.
- Rodar e simples: `inv test`.

Frase de impacto: *Os testes de invasao verificam que um funcionario nao acessa o comprovante de outro - retorna 403.*

Demo sugerida: rodar `inv test` ao vivo e mostrar os 76 testes passando em verde.

Criterio 7 - Qualidade de Codigo, Organizacao e Boas Praticas

Quem fala: Rychard Chaves | **Arquivos:** pyproject.toml, .env.*, E Ponto/utis/

Fala: “Fechando: o projeto segue boas praticas de engenharia de ponta a ponta.”

O que mostrar:

- .env por ambiente (dev/test/prod) + .example versionados, sem segredos: SECRET_KEY, banco e e-mail nunca ficam no codigo.
- pyproject.toml (PEP 621): dependencias separadas em dependencies, dev (black, flake8) e test (pytest, pytest-cov).
- Automacao com Invoke: `inv run`, `inv test`, `inv lint`, `inv format`, `inv zip`.
- Tratamento de erros: handlers 403/404/500 + `db.session.rollback()` no 500. Logging via `app.logger`.
- Seguranca: Bcrypt, 2FA (pyotp/qrcode), rate limiting, CSRF, AuditLog de acoes sensiveis.

- utils/ isola regras de negocio (clt, banco_horas, geo, nsr, hashing, afd/aej, pdf, excel) - views finas, logica testavel.
- Versionamento Git + .gitignore (venv, __pycache__, .db fora do repo). Ha um docs/RELATORIO_SEGURANCA.md.

Frase de impacto: *Cada decisao tem um porque documentado em comentario - o codigo foi escrito para ser lido e mantido.*

Encerramento (~30s) - Roberto Dias

“Em resumo: o E-Ponto é um sistema Flask com arquitetura de fábrica, 6 Blueprints, validação no servidor, 14 entidades com ORM e migrations, 76 testes automatizados, interface responsiva e conformidade legal REP-P. Tudo isso está no .zip entregue e detalhado no PDF de documentação. Obrigado - abrimos para perguntas.”

Checklist da entrega obrigatória

- .zip nomeado GrupoX_E-Ponto.zip (sem venv/ e __pycache__/).
- PDF de documentação com: título + integrantes; descrição e funcionalidades; diagrama ER; trechos técnicos comentados; explicação da arquitetura (App Factory); tecnologias + justificativa; passo a passo de execução; principais desafios.
- Desafios para citar: hash em cadeia, fuso horário UTC, geofencing, NSR atômico.
- Dica: já existem `analise_eponto.pdf` e `resumo_eponto.pdf` na raiz - podem servir de base para o PDF de documentação.